

BLE communication integration for automotive systems using esp32.

ESP32 Server – PC Client :

1. Introduction

Bluetooth Low Energy (BLE) is a wireless communication technology designed for low-power communication between electronic devices. It is widely used in embedded and automotive applications where devices need to exchange small amounts of data efficiently.

The objective is to establish BLE communication between the ESP32 as the BLE Server and the PC as the BLE Client.

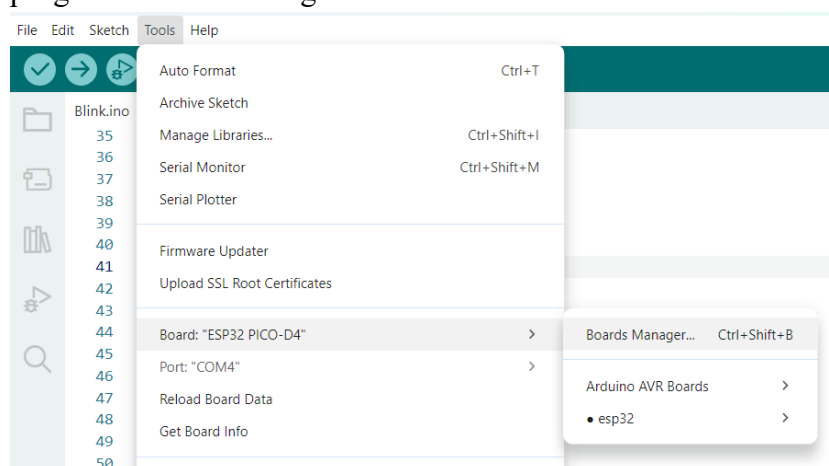
2. BLE Environment Setup

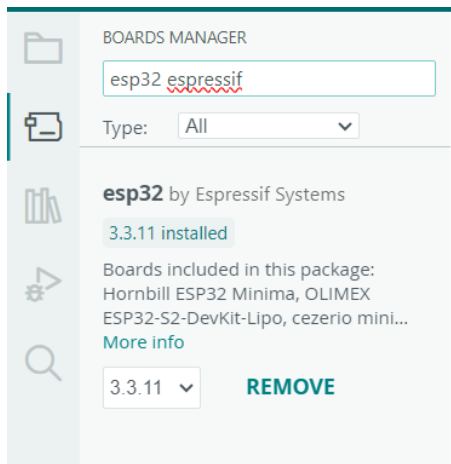
2.1 Required Hardware

- ESP32
- USB cable
- Two LEDs
- 2 Resistors
- Button
- PC

2.2 Installing ESP32 Board Support

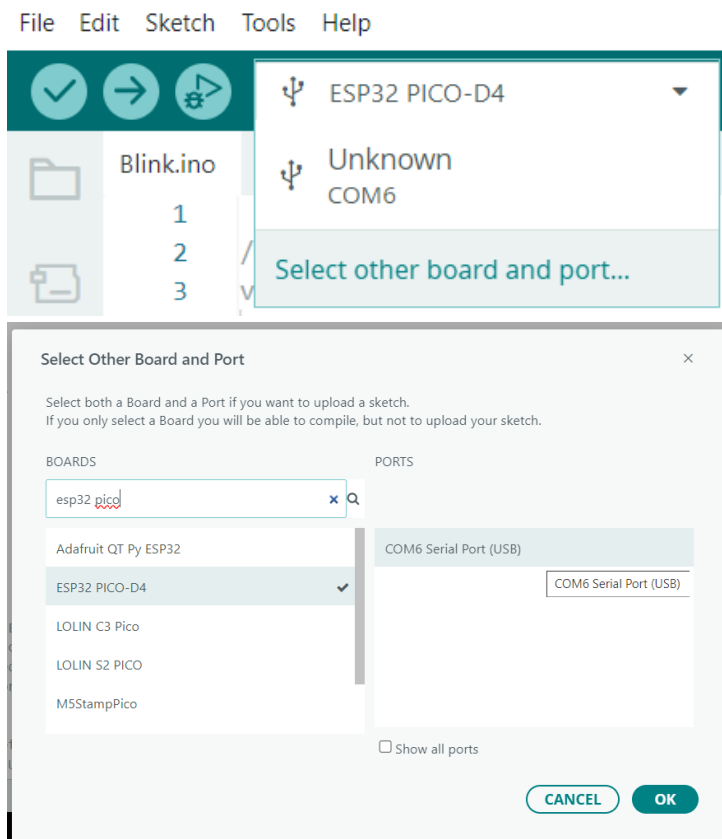
The ESP32 board support package was installed in the Arduino IDE to allow the ESP32 to be programmed and configured.





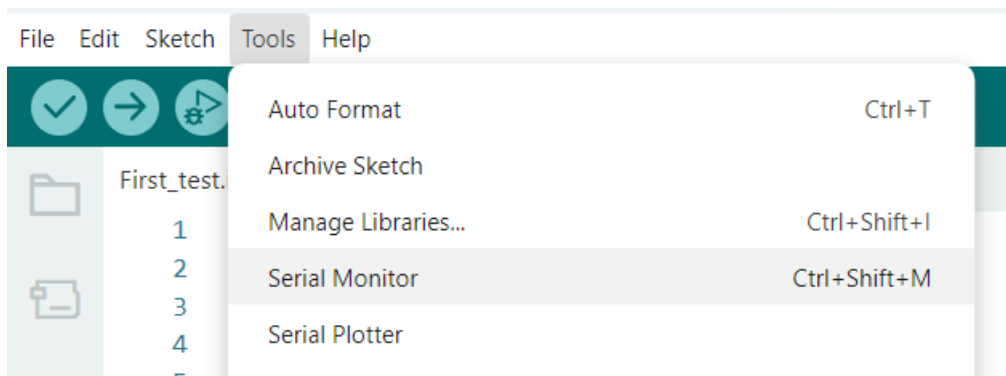
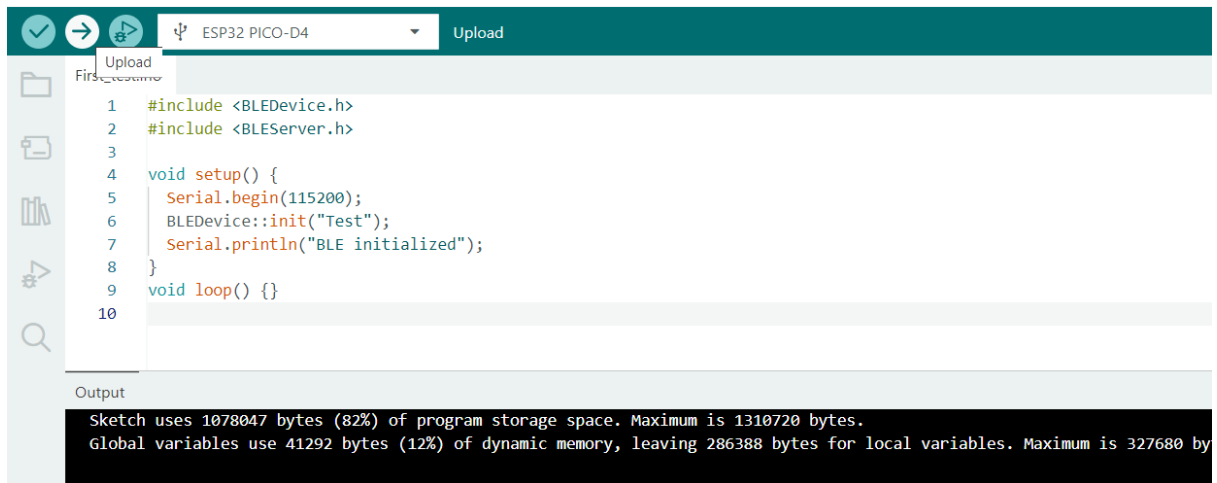
2.3 Selecting the Board and Port

The ESP32 board and its corresponding serial port were selected in Arduino IDE.



2.4 Testing the ESP32

Before implementing BLE communication, the ESP32 was tested to verify that the board could be successfully programmed and that communication through the Serial Monitor was working correctly.



3. Configuring the ESP32 as a BLE Server

The ESP32 is configured as a BLE Server that provides BLE services and characteristics that can be accessed by a connected BLE client.

```
BLEDevice::init("ESP32_LED_Control");
pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());
```

3.1 Connection Handling

When a client connects to the ESP32, the **onConnect()** callback is automatically executed. The first LED is switched ON to indicate that a BLE connection has been established.

```
void onConnect(BLEServer* pServer) {  
    digitalWrite(LED_CONNECTED, HIGH);  
    Serial.println("Device Connected! LED 1 is ON.");  
}
```

3.2 Disconnection Handling

When the client disconnects, the **onDisconnect()** callback is executed. The implementation switches both LEDs OFF and restarts advertising so that another device can connect.

```
void onDisconnect(BLEServer* pServer) {  
    // Force BOTH software states to off when connection drops  
    digitalWrite(LED_CONNECTED, LOW);  
    digitalWrite(LED_CONTROL, LOW);  
  
    Serial.println("Device Disconnected! Both LEDs are OFF.");  
  
    BLEDevice::startAdvertising();  
    Serial.println("Waiting for new connection...");  
}
```

```
Device Disconnected! Both LEDs are OFF.  
Waiting for new connection...
```

4. BLE Services and Characteristics

BLE application data is organized into services and characteristics. The service groups related functions, while the characteristic carries the exchanged data.

4.1 Custom Service/ Characteristic UUIDs

Create custom service UUID + characteristic UUID using UUID generator
(<https://www.uuidgenerator.net/>)

Version 4 UUID Generator

Generate a Version 4 UUID

Bulk Version 4 UUID Generation

How Many?

1

Generate

[Download to a file](#)

Copy all as:

Text

JSON

96d0c797-a2a9-4aa5-a99e-b0d3cdf01bc8

```
#define SERVICE_UUID           "96d0c797-a2a9-4aa5-a99e-b0d3cdf01bc8"
#define CHARACTERISTIC_UUID    "beb5483e-36e1-4688-b7f5-ea07361b26a8"
```

4.2 BLE Service

```
BLEService *pService = pServer->createService(SERVICE_UUID);
```

The ESP32 exposes the standard Generic Access and Generic Attribute services together with the custom application service.

Generic Access Service: provides fundamental information about the BLE device, including information such as the device name.

Generic Attribute Service: a mandatory service related to the organization and management of the device's GATT structure

Service Name: **GenericAttribute**

Service UUID: **00001801-0000-1000-8000-00805f9b34fb**

Characteristic Name: **ServiceChanged** - Characteristic Short UUID: **0x2A05** - User Description: -

Handle: **2** - **0x00000002** - Value: **Read Not Permitted**

Service Name: **GenericAccess**

Service UUID: **00001800-0000-1000-8000-00805f9b34fb**

Characteristic Name: **DeviceName** - Characteristic Short UUID: **0x2A00** - User Description: - Handle:

21 - **0x00000015** - Value: **ESP32_LED_Control**

Characteristic Name: **Appearance** - Characteristic Short UUID: **0x2A01** - User Description: - Handle:

23 - **0x00000017** - Value: **00-00**

Characteristic Name: **CentralAddressResolution** - Characteristic Short UUID: **0x2AA6** - User

Description: - Handle: **25** - **0x00000019** - Value: **00**

Service Name: **96d0c797-a2a9-4aa5-a99e-b0d3cdf01bc8**

Service UUID: **96d0c797-a2a9-4aa5-a99e-b0d3cdf01bc8**

Characteristic Name: **beb5483e-36e1-4688-b7f5-ea07361b26a8** - Characteristic Short UUID: - User

Description: - Handle: **41** - **0x00000029** - Value: **UNLOCK**

4.3 BLE Characteristic

A custom characteristic was created inside the custom service.

```

pCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID,
    BLECharacteristic::PROPERTY_WRITE |
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_NOTIFY
);

pCharacteristic->addDescriptor(new BLE2902());
pCharacteristic->setCallbacks(new MyCharacteristicCallbacks());
pCharacteristic->setValue("LOCK");

```

Properties: Read, Write, Notify

☒ Use Windows notifications

5. BLE Advertising

BLE advertising allows the ESP32 to announce its presence before a connection is established.

The ESP32 advertises the custom service UUID with scan response enabled and then starts BLE advertising.

```

BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();

```

```

Serial.println("BLE is active and broadcasting. Waiting for connection...");

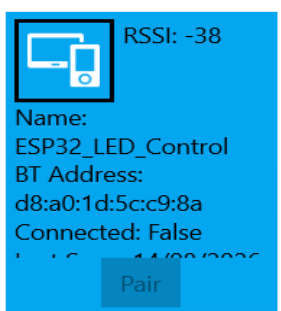
```



6. Discovering the ESP32 from the PC

After advertising starts, the PC can discover the ESP32 using its BLE device name.

During testing, the BLE device information was explored from the application “**Bluetooth LE Explorer**”.



8. Characteristic Read / Write / Notify Operation

When the PC writes data to the characteristic, the **onWrite()** callback is executed.

```
void onWrite(BLECharacteristic *pCharacteristic) {  
    String value = pCharacteristic->getValue();  
    Serial.print("Write Received: ");  
    Serial.print(value);  
    Serial.println("");  
    value.trim(); // Clean up any invisible windows carriage returns
```

If the received character is '1', LED 2 is switched ON, the value to be read is “UNLOCK”. If the received character is '0', LED 2 is switched OFF, the value to be read is “LOCK”. For both cases, the user will receive a notification either “LOCK” or “UNLOCK”.

```
if (value == "1") {  
    digitalWrite(LED_CONTROL, HIGH);  
    Serial.println("Received '1' - LED 2 ON");  
    // Update the value so a READ operation returns "UNLOCK"  
    pCharacteristic->setValue("UNLOCK");  
    Serial.println("Value was read by client: UNLOCK");  
    // Push the NOTIFY operation to the client  
    pCharacteristic->notify();  
    Serial.println("-> Notification successfully pushed: UNLOCK");  
}  
else if (value == "0") {  
    digitalWrite(LED_CONTROL, LOW);  
    Serial.println("Received '0' - LED 2 OFF");  
    pCharacteristic->setValue("LOCK");  
    Serial.println("Value was read by client: LOCK");  
    pCharacteristic->notify();  
    Serial.println("-> Notification successfully pushed: LOCK");  
}  
else {  
    Serial.println("Error: Command not recognized. Send '1' or '0'.");  
}
```

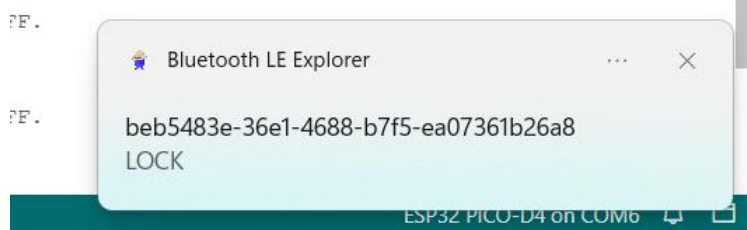
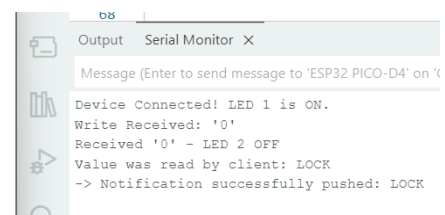
Read Value:

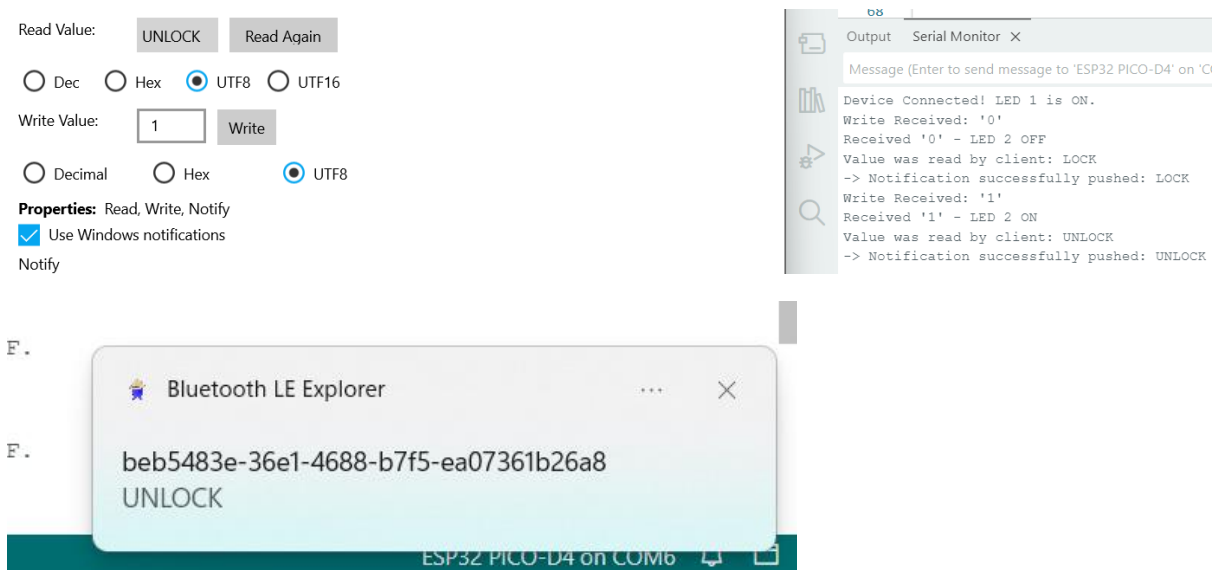
☐ Dec ☐ Hex ☒ UTF8 ☐ UTF16

Write Value:

☐ Decimal ☐ Hex ☒ UTF8

Properties: Read, Write, Notify





9. Security (Passkey Entry)

```
class MySecurityCallbacks : public BLESecurityCallbacks {
    void onPassKeyNotify(uint32_t pass_key) {
        Serial.print(">>> PAIRING PIN: ");
        Serial.printf("%06d\n", pass_key);
    }
    bool onSecurityRequest() { return true; }

    void onAuthenticationComplete(esp_ble_auth_cmpl_t cmpl) {
        if (cmpl.success) Serial.println("Pairing successful! Encrypted.");
        else Serial.println("Pairing failed!");
    }
    uint32_t onPassKeyRequest() { return 0; }
    bool onConfirmPIN(uint32_t pass_key) { return true; }
};
```

